

Solving Recurrence

- 1. Iterative method**
- 2. Recursion tree method**
- 3. Substitution method**
- 4. Master Method**

Iterative method

For simplicity assume:

$$n = 2^k$$

$$\Rightarrow k = \log n$$

$$\Rightarrow T(n) = 2^k + 2^{k-1} + \dots + 2^2 + 2^1 + 2^0$$

$$\Rightarrow T(n) = (2^{k+1} - 1) / (2 - 1)$$

$$\Rightarrow T(n) = 2^{k+1} - 1$$

$$\Rightarrow T(n) = 2 \cdot 2^k - 1$$

$$\Rightarrow T(n) = 2n - 1$$

$$\Rightarrow T(n) = O(n)$$

$$T(n) = 2T(n/2) + 1 \quad \text{when } n > 1$$

$$T(n) = 1 \quad \text{when } n = 1$$

$$T(n) = 2T(n/2) + 1$$

$$= 2 \{ 2T(n/4) + 1 \} + 1$$

$$= 4T(n/4) + 2 + 1$$

$$= 4 \{ 2T(n/8) + 1 \} + 2 + 1$$

$$= 8T(n/8) + 4 + 2 + 1$$

.....

.....

$$= 2^k T(n/2^k) + 2^{k-1} T(n/2^{k-1}) + \dots + 4 + 2 + 1.$$

Iterative method

$$T(n) = T(n/3) + O(n)$$

when $n > 1$

$$T(n) = 1$$

when $n = 1$

Iterative method

$$T(n) = T(n/3) + O(n) \quad \text{when } n > 1$$

$$T(n) = 1 \quad \text{when } n = 1$$

$$T(n) = T(n/3) + O(n)$$

$$\Rightarrow T(n) = T(n/3^2) + O(n/3) + O(n)$$

$$\Rightarrow T(n) = T(n/3^3) + O(n/3^2) + O(n/3) + O(n)$$

$$\Rightarrow T(n) = T(n/3^4) + O(n/3^3) + O(n/3^2) + O(n/3) + O(n)$$

$$\Rightarrow T(n) = T(n/3^k) + O(n/3^{k-1}) + \dots + O(n/3) + O(n)$$

For Simplicity assume

$$n = 3^k$$

$$\Rightarrow k = \log_3 n$$

$$\Rightarrow T(n) \leq T(1) + c n/3^{k-1} + c n/3^2 + c n/3 + c n$$

$$\Rightarrow T(n) \leq 1 + \{ c n/3^{k-1} + c n/3^2 + c n/3 + c n \}$$

$$\Rightarrow T(n) \leq 1 + c n \{ 1/(1-1/3) \}$$

$$\Rightarrow T(n) \leq 1 + 3/2 c n$$

$$T(n) = O(n)$$

Solving Recurrence Relation by Recursion Tree

More on recursion tree

$$\begin{array}{ll} T(n) = 1 & n = 1 \\ T(n) = T(n/2) + 1 & n > 1 \end{array}$$

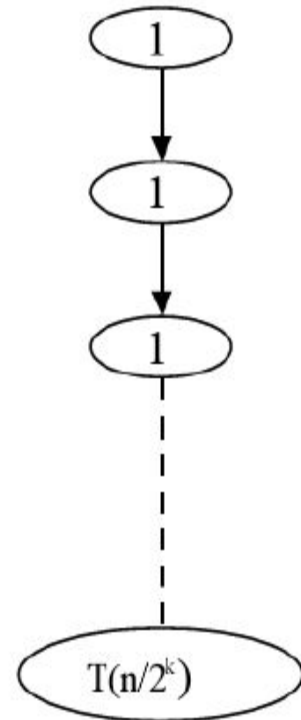
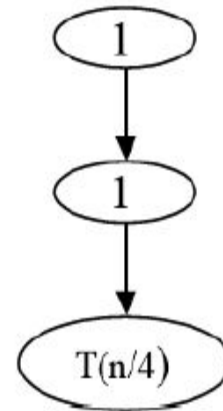
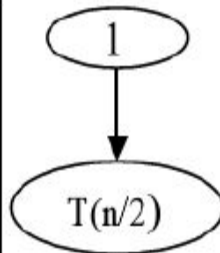
Solution: Drawing recursion tree from given relation and accumulating the cost,

Cost at each level = 1

For simplicity, assume $n = 2^k$

Then $k = \log n$

Total cost = $1 + 1 + 1 + \dots$ upto $\log n$ terms = $O(\log n)$



Recursion Tree

$$\begin{array}{ll} T(n) = 1 & n = 1 \\ T(n) = 4T(n/2) + n & n > 1 \end{array}$$

Recursion Tree

$$T(n) = 1 \quad n = 1$$

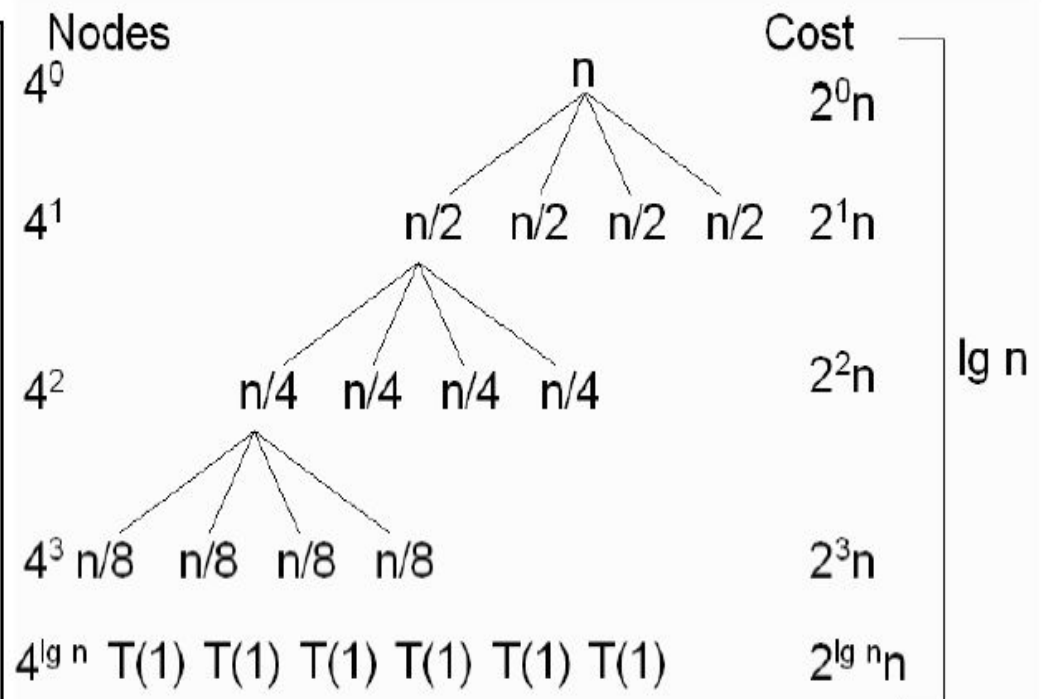
$$T(n) = 4T(n/2) + n \quad n > 1$$

Solution:

→ Bottom level has $n=1$ size problems or $T(1)$

→ Assuming $n = 2^k$, so $k = \log n$.

$$\begin{aligned} \rightarrow \text{Total Cost: } & 2^0n + 2^1n + 2^2n + \dots + 2^{k-1}n + 2^kn \\ &= n(1 + 2 + 4 + \dots + 2^{k-1} + 2^k) \\ &= n(2^{k+1} - 1)/(2 - 1) \text{ [Geometric ser.]} \\ &= n(2^{k+1} - 1) \\ &\leq n2^{k+1} \\ &= 2n \cdot 2^k \\ &= 2n \cdot n \\ &= O(n^2) \end{aligned}$$

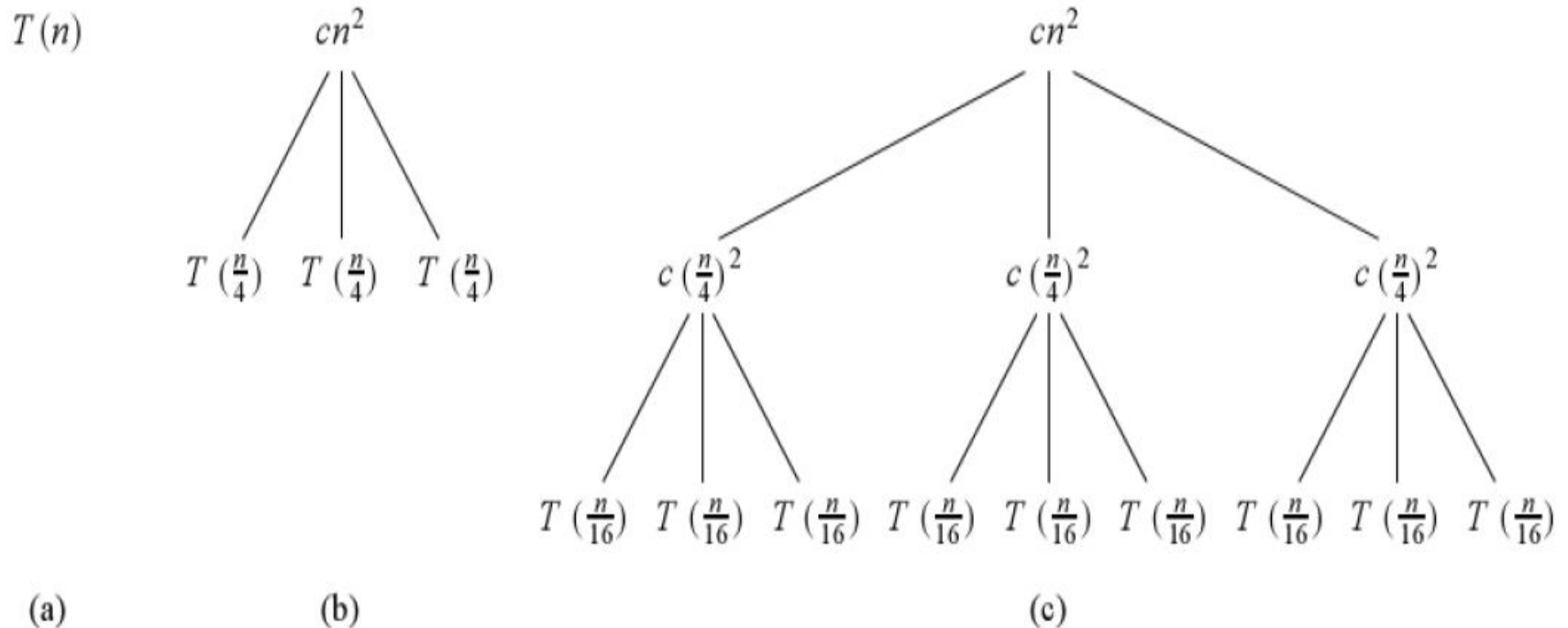


Example III:

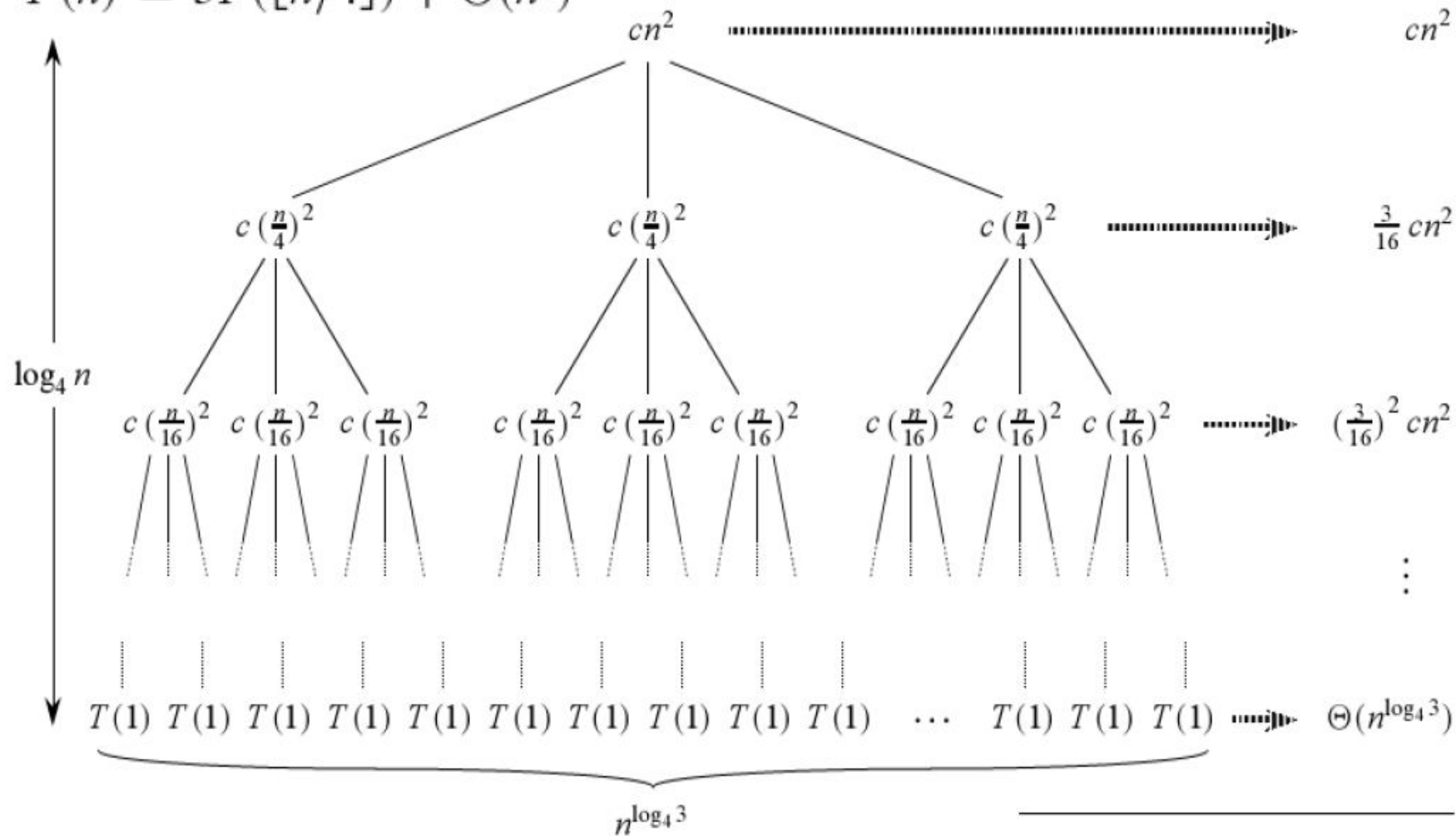
$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$$

Example III:

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$$



$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$$



(d)

Total: $O(n^2)$

Total cost

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\ &= O(n^2) . \end{aligned}$$

Case is of decreasing geometric series, so use of

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x} .$$

Substitution Method

1. Guess the form of the solution, using unknown constants.
2. Use induction to find the constants & verify the solution.

$$T(n) = 1 \qquad n=1$$

$$T(n) = 4T(n/2) + n \qquad n>1$$

#1

Substitution Method

$$T(n) = 1 \quad n=1$$

$$T(n) = 4T(n/2) + n \quad n>1$$

Let's Guess : $T(n) = O(n^3)$.

More specifically:

$T(n) \leq cn^3$, for all large enough n .

Prove by strong induction on n .

Assume: $T(k) \leq ck^3$ for $\forall k < n$.

Show: $T(n) \leq cn^3$ for $\forall n > n_0$.

Base case,

For $n=1$:

$$T(n) = 1$$

Definition

$$1 \leq c$$

Choose large enough c for conclusion

Inductive case, $n>1$:

$$T(n) = 4T(n/2) + n \quad \text{Definition.}$$

$$\leq 4c \cdot (n/2)^3 + n \quad \text{Induction.}$$

$$= c/2 \cdot n^3 + n \quad \text{Algebra.}$$

$$T(n) \leq c/2 \cdot n^3 + n$$

$$= cn^3 - (c/2 \cdot n^3 - n)$$

$$\leq cn^3$$

Proved:

$$T(n) \leq 2n^3 \text{ for } \forall n > 0$$

$$\text{Thus, } T(n) = O(n^3).$$

from before.

Algebra.

Since $n>0$, if $c \geq 2$

#2

Substitution Method

$$T(n) = 1 \quad n=1$$

$$T(n) = 4T(n/2) + n \quad n>1$$

Guess: $T(n) = O(n^2)$.

Same recurrence, but now try tighter bound.

More specifically:

$$T(n) \leq cn^2 \text{ for } \forall n > n_0.$$

Assume $T(k) \leq ck^2$, for $\forall k < n$.

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &\leq 4c \cdot (n/2)^2 + n \\ &= cn^2 + n \end{aligned}$$

Not $\leq cn^2$!

Problem is that the constant isn't shrinking.

Solution: Use a tighter guess & inductive hypothesis.

Subtract a lower-order term – a common technique.

Guess:

$$T(n) \leq cn^2 - dn \text{ for } \forall n > 0$$

Assume $T(k) \leq ck^2 - dk$, for $\forall k < n$. Show $T(n) \leq cn^2 - dn$.

Base case, $n=1$

$$T(n) = 1$$

Definition.

$$1 \leq c-d$$

Choosing c, d appropriately.

Inductive case, $n>1$:

$$T(n) = 4T(n/2) + n$$

Definition.

$$\leq 4(c(n/2)^2 - d(n/2)) + n$$

Induction.

$$= cn^2 - 2dn + n$$

Algebra.

$$= cn^2 - dn - (dn - n)$$

Algebra.

$$\leq cn^2 - dn$$

Choosing $d \geq 1$.

$$T(n) \leq 2n^2 - dn \text{ for } \forall n > 0$$

$$\text{Thus, } T(n) = O(n^2).$$

Changing Variables:

Sometimes a little algebraic manipulation can make a unknown recurrence similar to one we have seen

Consider the example

$$T(n) = 2T(\lfloor n^{1/2} \rfloor) + \log n$$

Looks Difficult: Rearrange like

$$\text{Let } m = \log n \Rightarrow n = 2^m$$

Thus,

$$T(2^m) = 2T(2^{m/2}) + m$$

Again let

$$S(m) = T(2^m) \quad S(m) = 2S(m/2) + m$$

We can show that

$$S(m) = O(m \log m)$$

$$\Rightarrow T(n) = T(2^m) = S(m) = O(m \log m) = O(\log n \log \log n)$$

Solving Recurrence by Master Method

Master method provides cookbook method for solving recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$, $b > 1$, and $f(n) > 0$

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ can be bounded asymptotically as follows.

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

To use the master method, we simply determine which case (if any) of the master theorem applies and write down the answer.

As a first example, consider

$$T(n) = 9T(n/3) + n.$$

For this recurrence, we have $a = 9$, $b = 3$, $f(n) = n$, and thus we have that $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9 - \epsilon})$, where $\epsilon = 1$, we can apply case 1 of the master theorem and conclude that the solution is $T(n) = \Theta(n^2)$.

Now consider

$$T(n) = T(2n/3) + 1,$$

in which $a = 1$, $b = 3/2$, $f(n) = 1$, and $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Case 2 applies, since $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$, and thus the solution to the recurrence is $T(n) = \Theta(\lg n)$.

$$T(n) = 3T(n/4) + n \lg n ,$$

we have $a = 3$, $b = 4$, $f(n) = n \lg n$, and $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. Since $f(n) = \Omega(n^{\log_4 3 + \epsilon})$, where $\epsilon \approx 0.2$, case 3 applies if we can show that the regularity condition holds for $f(n)$. For sufficiently large n , $af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n)$ for $c = 3/4$. Consequently, by case 3, the solution to the recurrence is $T(n) = \Theta(n \lg n)$.

The master method does not apply to the recurrence

$$T(n) = 2T(n/2) + n \lg n ,$$

even though it has the proper form: $a = 2$, $b = 2$, $f(n) = n \lg n$, and $n^{\log_b a} = n$. It might seem that case 3 should apply, since $f(n) = n \lg n$ is asymptotically larger than $n^{\log_b a} = n$. The problem is that it is not *polynomially* larger. The ratio $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ is asymptotically less than n^ϵ for any positive constant ϵ . Consequently, the recurrence falls into the gap between case 2 and case 3.

